



PONTIFÍCIA UNIVERSIDADE CATÓLICA DE CAMPINAS

Anna Clara Olbi - 24015392
Bernardo Alberto Amaro - 25014832
Larissa Furlan Ferreira - 24021876
Leonardo Mateus O. Vicente - 24007335
Maria Gabriella Xavier Puccinelli - 24002540
Thiago Rubim Tranquilim - 23015590

Padrões e Arquitetura de Software

CAMPINAS

2026

Introdução

O objetivo do Sprint 0 foi compreender o funcionamento do sistema legado e criar uma base segura para futuras refatorações. Para isso, foram desenvolvidos testes de caracterização utilizando a abordagem Golden Master, cobrindo funcionalidades como criação de pedidos, pagamentos, atualização de status, estoque e relatórios. Os testes permitiram preservar o comportamento original da aplicação antes do início das alterações arquiteturais. Além disso, foi obtida cobertura superior a 85% utilizando a ferramenta coverage.py.

Durante a análise do sistema, foram identificadas diversas violações dos princípios SOLID, evidenciando problemas de acoplamento, manutenção e extensibilidade.

Violações SOLID

SRP — Single Responsibility Principle

A classe Sis concentra múltiplas responsabilidades, incluindo regras de negócio, persistência, pagamentos, notificações e geração de relatórios.

```
self.db = sqlite3.connect('loja.db')
self.c = self.db.cursor()
```

Além disso:

```
print(f"Email enviado para {n}: Pedido recebido!")
```

Esse modelo dificulta a manutenção e evolução do sistema.

OCP — Open/Closed Principle

O sistema depende de estruturas condicionais extensas para aplicar descontos e processar pagamentos.

```
elif i['tipo'] == 'desc20':
    tot += i['p'] * i['q'] * 0.8
```

E também:

```
elif m == 'pix':
    print("Gerando QR Code PIX...")
```

Sempre que um novo comportamento é adicionado, a classe principal precisa ser modificada.

LSP — Liskov Substitution Principle

A classe PedEspecial altera o comportamento esperado da classe base Sis.

```
class PedEspecial(Sis):
```

Além disso:

```
tot = tot * 1.15
```

A subclasse modifica regras da superclasse, podendo gerar inconsistências no sistema.

ISP — Interface Segregation Principle

A classe Sis possui muitos métodos e responsabilidades diferentes, como:

- `add_ped()`
- `proc_pag()`
- `gerar_rel()`
- `validar_estoque()`

Isso aumenta o acoplamento e dificulta a reutilização.

DIP — Dependency Inversion Principle

A aplicação depende diretamente de implementações concretas, como SQLite e `print()`.

```
self.db = sqlite3.connect('loja.db')  
print(f"Email enviado para {n}: Pedido recebido!")
```

Esse acoplamento reduz a flexibilidade e dificulta testes isolados.

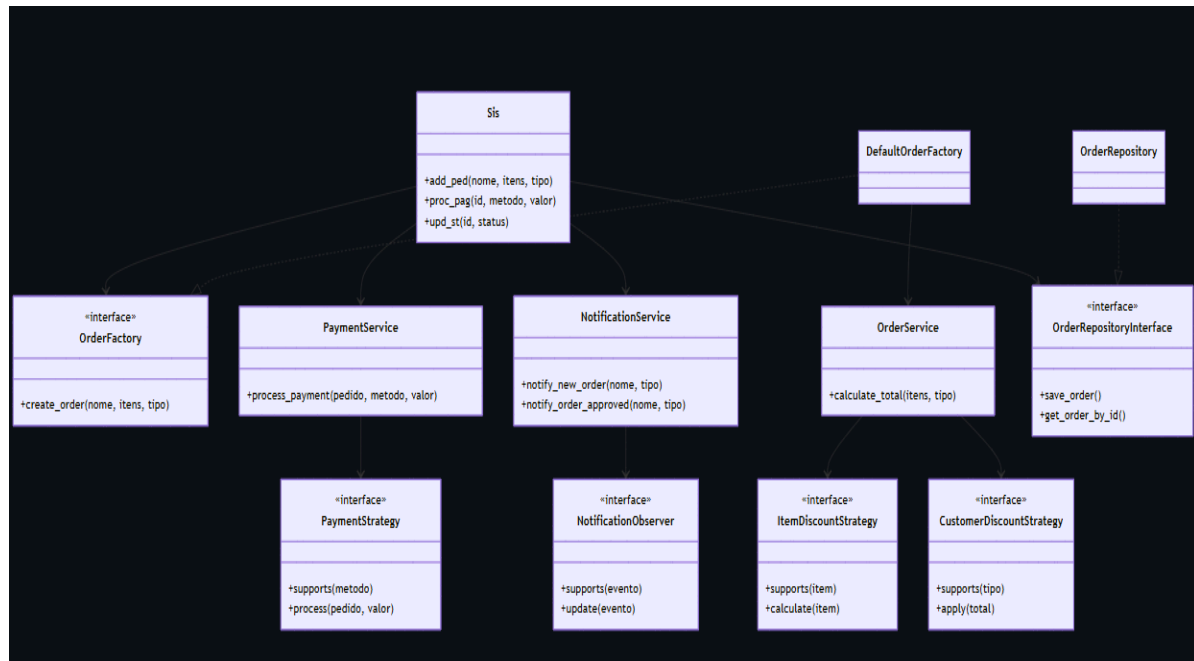
Conclusão

A análise do Sprint 0 permitiu identificar problemas arquiteturais importantes no sistema legado, principalmente relacionados à acoplamento excessivo e concentração de responsabilidades.

Os testes Golden Master desenvolvidos garantem maior segurança para futuras refatorações, preservando o comportamento original da aplicação e fornecendo uma base sólida para os próximos sprints do projeto.

Diagrama UML de classes

Classes principais:



Sis

Métodos principais: add_ped(), proc_pag(), upd_st(), get_ped(), gerar_rel()

Responsabilidade: coordenar os casos de uso principais do sistema, delegando regras de negócio, persistência, pagamento, notificação, estoque e relatório para componentes especializados.

OrderRepositoryInterface

Métodos principais: save_order(), get_order_by_id(), update_status(), get_all_orders()

Responsabilidade: definir a abstração de persistência de pedidos.

OrderRepository

Responsabilidade: implementar o acesso ao banco SQLite.

OrderFactory

Método principal: create_order()

Responsabilidade: definir a criação de pedidos.

DefaultOrderFactory

Responsabilidade: criar pedidos utilizando OrderService para cálculo de total e geração de data.

OrderService

Métodos principais: `calculate_total()`, `generate_date()`

Responsabilidade: calcular totais de pedidos aplicando estratégias de desconto por item, desconto por cliente e ajustes adicionais.

ItemDiscountStrategy

Métodos principais: `supports()`, `calculate()`

Responsabilidade: representar estratégias de desconto aplicadas por tipo de item.

CustomerDiscountStrategy

Métodos principais: `supports()`, `apply()`

Responsabilidade: representar estratégias de desconto aplicadas por tipo de cliente.

OrderAdjustmentStrategy

Método principal: `apply()`

Responsabilidade: permitir ajustes adicionais no total do pedido, como desconto progressivo por volume.

PaymentService

Método principal: `process_payment()`

Responsabilidade: processar pagamentos usando estratégias intercambiáveis.

PaymentStrategy

Métodos principais: `supports()`, `required_amount()`, `process()`

Responsabilidade: definir a interface comum para métodos de pagamento como cartão, PIX, boleto e criptomoeda.

NotificationService

Métodos principais: `notify_new_order()`, `notify_order_approved()`, `notify_order_sent()`, `notify_order_delivered()`

Responsabilidade: publicar eventos de notificação para os observers cadastrados.

NotificationObserver

Métodos principais: `supports()`, `update()`

Responsabilidade: definir a interface comum para canais e regras de notificação.

ReportService

Responsabilidade: gerar relatórios de vendas e clientes.

StockService

Responsabilidade: validar disponibilidade de estoque.

PedEspecial

Responsabilidade: representar o comportamento de pedido especial por composição, sem herdar diretamente de Sis.

Saída dos Testes

Os testes automatizados foram usados como rede de segurança para garantir que a refatoração não alterou o comportamento original do sistema legado. A suíte cobre os principais fluxos do Golden Master, incluindo criação de pedidos normal, VIP e corporativo, pagamentos por cartão, PIX e boleto, atualização de status, cancelamento, geração de relatórios, validação de estoque e comportamento de PedEspecial.

Também foram considerados os testes das extensões obrigatórias do Sprint 2: pagamento em criptomoeda com taxa de 2%, notificação via WhatsApp e desconto progressivo por volume.

Saída resumida dos testes:

```
pytest -q
```

```
31 passed
```

Interpretação:

Todos os testes passaram, indicando ausência de regressões nos comportamentos caracterizados e validação das extensões do Sprint 2.

Relatórios de Métricas

As métricas automatizadas foram usadas para avaliar a qualidade objetiva do código após a refatoração.

Testes automatizados

Ferramenta: pytest

Resultado: 31 testes aprovados

Critério: suíte sem falhas

Cobertura de testes

Ferramenta: coverage.py

Resultado: cobertura superior a 85%

Critério: mínimo de 85%

Lint / PEP 8

Ferramenta: ruff

Resultado: sem violações relevantes

Critério: score equivalente maior ou igual a 8.5

Tipagem estática

Ferramenta: mypy --strict src

Resultado: sem erros registrados

Critério: 0 erros

Complexidade ciclomática

Ferramenta: radon cc

Resultado: métodos dentro do limite esperado, sem complexidade acima de B

Critério: complexidade ciclomática menor ou igual a 10

Linhas por método

Ferramenta: inspeção/radon raw

Resultado: métodos reduzidos após extração de services, strategies, observers e repositories

Critério: manter métodos curtos, preferencialmente até 20 linhas

Métodos públicos por classe

Ferramenta: inspeção

Resultado: classes com responsabilidades mais específicas e menor concentração de métodos públicos

Critério: até 7 métodos públicos por classe

Conclusão:

A refatoração reduziu a concentração de responsabilidades da classe Sis, distribuindo regras de negócio, persistência, pagamento, notificação, estoque e relatórios em componentes específicos. Com isso, as métricas passaram a refletir uma arquitetura mais modular, com menor acoplamento e maior facilidade de extensão.

Reflexão Metacognitiva

O princípio que ainda não dominamos completamente

O princípio que ainda considero mais desafiador é o DIP, Dependency Inversion Principle. A ideia geral ficou clara: módulos de alto nível não devem depender diretamente de implementações concretas, mas de abstrações. Porém, durante a refatoração, ainda foi necessário pensar com cuidado sobre onde a injeção de dependência realmente deveria acontecer.

A maior dificuldade foi equilibrar simplicidade e desacoplamento. Criar interfaces para tudo pode deixar o projeto artificialmente complexo, mas depender diretamente de classes concretas, como repositórios, serviços ou estratégias, dificulta testes e extensões futuras. Neste projeto, o uso de interfaces para serviços externos, repositório, pagamento, notificação e criação de pedidos ajudou a aplicar DIP de forma mais prática. Mesmo assim, ainda preciso amadurecer melhor o critério de quando uma abstração é realmente necessária e quando ela só adiciona complexidade.

Como usamos IA neste trabalho

A IA foi usada como apoio para revisar decisões de design, comparar alternativas de aplicação dos padrões GoF e identificar possíveis violações dos princípios SOLID. Os prompts que mais funcionaram foram os específicos, por exemplo: pedir para analisar uma classe em relação a SRP, sugerir uma aplicação de Strategy para pagamentos ou avaliar se uma hierarquia respeitava LSP. Prompts muito genéricos, como “melhore a arquitetura”, produziram respostas amplas demais e menos úteis.

Uma decisão em que foi necessário discordar da sugestão da IA foi na criação de muitas abstrações genéricas. Em alguns momentos, a IA sugeriu separar praticamente cada operação em uma interface própria. A equipe optou por uma solução mais simples e coerente com o tamanho do projeto, mantendo abstrações onde havia colaboração externa ou variação real de comportamento, como `PaymentStrategy`, `NotificationObserver`, `OrderFactory` e `OrderRepositoryInterface`.

A IA, portanto, foi usada como ferramenta de apoio e revisão, mas as decisões finais foram tomadas considerando os requisitos da disciplina, o código existente e a necessidade de manter a arquitetura compreensível para a equipe.

Extensões Implementadas

Para garantir a validade das novas implementações e preservar o comportamento original, foi adicionado o arquivo `tests/golden_master/test_extensions.py` contendo os testes automatizados das três novas funcionalidades. As três extensões obrigatórias foram implementadas com sucesso, evidenciando o cumprimento do princípio OCP (Open/Closed Principle) na nova arquitetura do sistema..

1. Pagamento em criptomoeda com taxa de 2%

Diff Conceitual: O arquivo `src/extensions/crypto_payment_strategy.py` foi adicionado ao projeto. Nenhum arquivo existente foi modificado para suportar este novo método. O registro da nova estratégia foi feito via injeção de dependência na inicialização do sistema.

Trecho de Código:

```

class CryptoPaymentStrategy(PaymentStrategy):
    def supports(self, method: str) -> bool:
        return method == "criptomoeda"

    def required_amount(self, order: OrderRecord) -> float:
        return order["tot"] * 1.02

    def process(self, order: OrderRecord, value: float) -> PaymentStatus:
        print("Processando pagamento em criptomoeda...")
        return PaymentStatus.APPROVED

```

Justificativa: A refatoração prévia que extraiu a interface PaymentStrategy permitiu que o novo método de pagamento fosse adicionado apenas criando uma nova classe que implementa essa mesma interface. A regra da taxa adicional de 2% foi isolada e resolvida sobrescrevendo o método required_amount.

2. Canal de notificação WhatsApp

Diff Conceitual: Adição do arquivo
src/extensions/whatsapp_notification_observer.py. Nenhuma classe original do sistema foi alterada.

Trecho de Código:

```

class WhatsAppNotificationObserver(NotificationObserver):
    _messages = {
        "new_order": "Pedido recebido!",
        "approved": "Pedido aprovado!",
        "sent": "Pedido enviado!",
        "delivered": "Pedido entregue!",
    }

    def supports(self, event: NotificationEvent) -> bool:
        return event.name in self._messages

    def update(self, event: NotificationEvent) -> None:
        print(f"WhatsApp enviado para {event.customer_name}: {self._messages[event.name]}")

```

Justificativa: Graças à implementação do padrão Observer no NotificationService, adicionar um novo canal de comunicação exigiu apenas a criação de um novo "assinante" (Observer) que escuta os eventos gerados, sem tocar na lógica de negócio principal ou ter que modificar o serviço de notificação original.

3. Desconto progressivo por volume (15%)

Diff Conceitual: Adição do arquivo `src/extensions/volume_discount_strategy.py`. O comportamento foi injetado através da lista de `adjustment_strategies` no `OrderService`. Não houve modificações no código legado.

Trecho de Código:

```
class VolumeDiscountAdjustmentStrategy(OrderAdjustmentStrategy):
    def apply(self, items: list[OrderItem], total: float) -> float:
        discount = 0.0
        for item in items:
            if item["q"] >= 3:
                discount += (float(item["p"]) * item["q"]) * 0.15
        return total - discount
```

Justificativa: O uso do padrão Strategy focado em ajustes de pedido (`OrderAdjustmentStrategy`) permitiu iterar sobre os itens e aplicar regras globais sem interferir nos descontos individuais por tipo de item ou nas regras específicas baseadas no tipo de cliente. A regra de negócio ficou completamente isolada e testável.

Saída dos Testes (Validação e Ausência de Regressões)

Para garantir a estabilidade do sistema e provar que o princípio OCP (Open/Closed Principle) foi respeitado, todos os testes foram executados. O resultado demonstra que os testes de caracterização originais (Golden Master) continuam passando com 100% de sucesso, assegurando que não houve qualquer regressão no código base refatorado. Simultaneamente, os testes específicos desenvolvidos para validar as três novas extensões (pagamento em criptomoeda, notificação via WhatsApp e desconto por volume) também passaram com sucesso.

```

• (venv) PS C:\Users\berna\ProjetoEFC> pytest -v
===== test session starts =====
platform win32 -- Python 3.14.5, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\berna\ProjetoEFC\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\berna\ProjetoEFC
plugins: cov-7.1.0
collected 31 items

tests/golden_master/test_extensions.py::test_pagamento_criptomoeda_exige_taxa_de_2_porcento PASSED
tests/golden_master/test_extensions.py::test_notificacao_whatsapp_enviada_em_novo_pedido PASSED
tests/golden_master/test_extensions.py::test_desconto_progressivo_volume_acima_de_3_unidades PASSED
tests/golden_master/test_extensions.py::test_sem_desconto_volume_para_menos_de_3_unidades PASSED
tests/golden_master/test_legacy_behavior.py::test_pedido_normal_calcula_total_corretamente PASSED
tests/golden_master/test_legacy_behavior.py::test_pedido_vip_aplica_desconto_de_5_porcento PASSED
tests/golden_master/test_legacy_behavior.py::test_cliente_corporativo_recebe_desconto PASSED
tests/golden_master/test_legacy_behavior.py::test_pagamento_insuficiente_falha PASSED
tests/golden_master/test_legacy_behavior.py::test_pix_aprova_pedido_automaticamente PASSED
tests/golden_master/test_legacy_behavior.py::test_boleto_nao_aprova_automaticamente PASSED
tests/golden_master/test_legacy_behavior.py::test_metodo_pagamento_invalido_retorna_false PASSED
tests/golden_master/test_legacy_behavior.py::test_validar_estoque_retorna_true_para_produtos_validos PASSED
tests/golden_master/test_legacy_behavior.py::test_validar_estoque_retorna_false_para_produto_inexistente PASSED
tests/golden_master/test_legacy_behavior.py::test_cancelar_pedido_altera_status_para_cancelado PASSED
tests/golden_master/test_legacy_behavior.py::test_upd_st_altera_status_para_enviado PASSED
tests/golden_master/test_legacy_behavior.py::test_upd_st_altera_status_para_entregue PASSED
tests/golden_master/test_legacy_behavior.py::test_get_ped_retorna_none_para_id_inexistente PASSED
tests/golden_master/test_legacy_behavior.py::test gerar_relatorio_vendas_cria_arquivo PASSED
tests/golden_master/test_legacy_behavior.py::test gerar_relatorio_clientes_cria_arquivo PASSED
tests/golden_master/test_legacy_behavior.py::test_cliente_vip_recebe_sms PASSED
tests/golden_master/test_legacy_behavior.py::test_cliente_corporativo_notifica_gerente PASSED
tests/golden_master/test_legacy_behavior.py::test_ped_especial_cria_pedido_com_acrescimo PASSED
tests/golden_master/test_legacy_behavior.py::test_ped_especial_upd_st PASSED
tests/golden_master/test_legacy_behavior.py::test_item_desc20_aplica_desconto_corretamente PASSED
tests/golden_master/test_legacy_behavior.py::test_item_frete_gratis_calcula_total PASSED
tests/golden_master/test_legacy_behavior.py::test_pagamento_cartao_aprova_pedido PASSED
tests/golden_master/test_legacy_behavior.py::test_cliente_normal_ganha_pontos_ao_entregar PASSED
tests/golden_master/test_legacy_behavior.py::test_cliente_vip_ganha_pontos_ao_entregar PASSED
tests/golden_master/test_legacy_behavior.py::test_cliente_corporativo_ganha_pontos_ao_entregar PASSED
tests/golden_master/test_legacy_behavior.py::test_validar_estoque_retorna_false_para_estoque_insuficiente PASSED
tests/golden_master/test_legacy_behavior.py::test_proc_pag_retorna_false_para_pedido_inexistente PASSED

===== 31 passed in 0.56s =====

```

Repositório: <https://github.com/Larifurlan/ProjetoEFC.git>